

Task 1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

- a) Write a C code to create a process using fork() system call.
- b) Read the user input as shell command.
- c) Use a system call exec() to execute the shell command.
- e) Display the PIDs of parent and child processes.

```
jagadeesh-14@Jagadeesh:~$ nano task01.c
jagadeesh-14@Jagadeesh:~$ gcc task01.c -o task01
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_INPUT 100
#define MAX_ARGS 20

int main() {
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("Enter a Linux command: ");
    fgets(input, sizeof(input), stdin);

    // Remove newline character
    input[strcspn(input, "\n")] = '\0';

    // Split the command into arguments
    int i = 0;
    args[i] = strtok(input, " ");

    while (args[i] != NULL && i < MAX_ARGS - 1) {
        i++;
        args[i] = strtok(NULL, " ");
    }

    pid_t pid = fork();

    if (pid < 0) {
        // Fork failed
        printf("Fork failed!\n");
    }
}
```

```

        return 1;
    }
    else if (pid == 0) {
        // Child process
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        // Execute command
        execvp(args[0], args);

        // Executes only if exec fails
        perror("Command execution failed");
        exit(1);
    }
    else {
        // Parent process
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID : %d\n", pid);
    }
}

```

1)Then press ctrl+o enter ctrl+x2)then gcc filename.c -o filename

```

jagadeesh-14@Jagadeesh:~$ nano task01.c
jagadeesh-14@Jagadeesh:~$ gcc task01.c -o task01
jagadeesh-14@Jagadeesh:~$

```

3)then ./filename

```

jagadeesh-14@Jagadeesh:~$ nano task01.c
jagadeesh-14@Jagadeesh:~$ gcc task01.c -o task01
jagadeesh-14@Jagadeesh:~$ ^C
jagadeesh-14@Jagadeesh:~$ ./task01
Enter a Linux command:

```

4)enter the linux command

```

jagadeesh-14@Jagadeesh:~$ ./task01
Enter a Linux command: ls

Child Process
Parent Process

```

```
Parent PID : 457
Child PID : 458
Child PID : 458
Parent PID: 457

Executing command: ls

PID.c  PIDs.c  PIDs.c.save  Sum  Sum.c  FIRST.C  linux.c  myfile.txt  notes.txt.save  os
os.c  process  process.c  prog  prog.c  task01  task01.c  task1.c  task1.c.save

Child process completed.
jagadeesh-14@Jagadeesh:~$
```

Child process completed.

Task 2

Using Linux terminal commands (uname, lscpu, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

An operating system is system software that controls all the hardware and software resources of a computer. Users do not communicate directly with the hardware. Instead, the operating system acts as a bridge between the user, applications, and the hardware

Linux Commands Used

1. uname

The uname command is used to display information about the Linux operating system. It provides details such as the operating system name, kernel version, machine architecture, and hostname. This command helps us identify the system on which Linux is running.

```
jagadeesh-14@Jagadeesh:~$ uname
Linux
```

2. lscpu

The lscpu command displays detailed information about the processor. It shows the CPU model, number of cores, number of threads, CPU architecture, and processor speed. This information helps us understand the hardware capabilities of the computer.

```
jagadeesh-14@Jagadeesh:~$ uname
Linux
jagadeesh-14@Jagadeesh:~$ lscpu
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:                39 bits physical, 48 bits virtual
```

```

Byte Order:                Little EndianCPU(s):                12
On-line CPU(s) list:       0-11
Vendor ID:                 GenuineIntel
Model name:               12th Gen Intel(R) Core(TM) i5-1235U
CPU family:               6
Model:                    154
Thread(s) per core:       2
Core(s) per socket:       6
Socket(s):                1
Stepping:                 4
BogoMIPS:                 4992.00
Flags:                    fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat
pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdtscp lm constant_tsc rep_good
nopl xtopology tsc_reliable nonstop_tsc cpuid tsc_known_freq pni pclmulqdq vmx sse3 fma cx16
pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand
hypervisor lahf_lm abm 3dnowprefetch ssbd ibrs ibpb stibp ibrs_enhanced tpr_shadow ept vpid
ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid rdseed adx smap clflushopt clwb
sha_ni xsaveopt xsavec xgetbv1 xsaves avx_vnni vnmi umip waitpkg gfni vaes vpclmulqdq rdpid
movdiri movdir64b fsrm md_clear serialize ibt flush_l1d arch_capabilities
Virtualization features:
Virtualization:           VT-x
Hypervisor vendor:        Microsoft
Virtualization type:      full
Caches (sum of all):
L1d:                      288 KiB (6 instances)

```

3. ps

The ps command lists the processes that are currently running in the system. It displays the Process ID (PID), Parent Process ID (PPID), user name, and the command associated with each process. This command is useful for monitoring and managing processes.

```

jagadeesh-14@Jagadeesh:~$ ps
  PID TTY          TIME CMD
  426 pts/0        00:00:00 bash
  476 pts/0        00:00:00 ps
jagadeesh-14@Jagadeesh:~$

```

4. top

The top command provides a real-time view of the system's performance. It continuously displays CPU usage, memory usage, running processes, and system load. It is commonly used to identify programs that consume a large amount of system resources.

```

top - 07:13:17 up 29 min, 1 user, load average: 0.00, 0.02, 0.00
Tasks: 22 total, 1 running, 21 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st

```

MiB Mem : 3757.6 total, 3188.2 free, 396.2 used, 247.8 buff/cache
MiB Swap: 1024.0 total, 1024.0 free, 0.0 used. 3361.5 avail Mem

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	21844	13080	9764	S	0.0	0.3	0:00.51	systemd
2	root	20	0	3180	2208	2072	S	0.0	0.1	0:00.01	init-systemd(Ub
7	root	20	0	3196	2128	2008	S	0.0	0.1	0:00.00	init
54	root	19	-1	50312	16212	15148	S	0.0	0.4	0:00.14	systemd-journal
100	root	20	0	25052	6540	5224	S	0.0	0.2	0:00.16	systemd-udev
116	systemd+	20	0	21344	13292	11164	S	0.0	0.3	0:00.08	systemd-resolve
117	systemd+	20	0	91036	8076	7104	S	0.0	0.2	0:00.10	systemd-timesyn
166	root	20	0	4244	2752	2496	S	0.0	0.1	0:00.00	cron
167	message+	20	0	9636	5452	4756	S	0.0	0.1	0:00.05	dbus-daemon
174	root	20	0	17976	8864	7820	S	0.0	0.2	0:00.05	systemd-logind
176	root	20	0	1830616	14320	11296	S	0.0	0.4	0:00.07	WSL-Pro-Service
181	syslog	20	0	222516	5688	4680	S	0.0	0.1	0:00.04	rsyslogd
193	root	20	0	3124	2040	1900	S	0.0	0.1	0:00.00	agetty
199	root	20	0	107036	23416	13964	S	0.0	0.6	0:00.10	unattended-upgr
294	root	20	0	6704	4740	3952	S	0.0	0.1	0:00.00	login
342	jagadee+	20	0	20332	11652	9520	S	0.0	0.3	0:00.12	systemd
343	jagadee+	20	0	21168	3572	1848	S	0.0	0.1	0:00.00	(sd-pam)
367	jagadee+	20	0	6080	5368	3700	S	0.0	0.1	0:00.01	bash
418	root	20	0	3188	1120	980	S	0.0	0.0	0:00.00	SessionLeader
420	root	20	0	3204	1260	1108	S	0.0	0.0	0:00.15	Relay(426)
426	jagadee+	20	0	6080	5420	3704	S	0.0	0.1	0:00.07	bash
479	jagadee+	20	0	9324	5776	3568	R	0.0	0.2	0:00.07	top